Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Современные методы защиты компьютерных систем»

Тема: «Разработка модулей ядра операционной системы для динамического перехвата системных вызовов с использованием технологии eBPF»

Выполнил:
Студент 4 курса
Группы ИИ-25
Жук Б.Д.
Проверила:
Хацкевич А.С.

Брест 2026

Цель работы: практическое освоение методов разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracepoints) и выявления аномального поведения процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection).

Ход работы:

Задание 1. Базовый перехват запуска процессов

Разработана программа на Python с использованием библиотеки BCC. В ядре Linux используется eBPF-программа, подключённая к tracepoint sys_enter_execve. При каждом вызове execve извлекаются PID процесса, UID пользователя и имя запускаемого файла.

```
#!/usr/bin/python3
from bcc import BPF
import time

bpf_text = """
#include <uapi/linux/ptrace.h>

struct data_t {
   u32 pid;
   u32 uid;
   char comm[256];
};

BPF_PERF_OUTPUT(events);

TRACEPOINT_PROBE(syscalls, sys_enter_execve) {
   struct data_t data = {};

   // Получение PID
   data.pid = bpf_get_current_pid_tgid() >> 32;

   // Получение UID
   data.uid = bpf_get_current_uid_gid();

   // Получение имени запускаемого файла
   bpf_probe_read_user_str(
      &data.comm,
      sizeof(data.comm),
      args->filename
   );

   // Передача события в User Space
   events.perf_submit(args, &data, sizeof(data));

   return 0;
}
"""
```

```
b = BPF(text=bpf_text)

print("[*] Мониторинг запущен. Ожидание вызовов execve... (Нажмите Ctrl+C для выхода)")
def print_event(cpu, data, size):
    event = b["events"].event(data)

    pid = event.pid
    uid = event.uid
    comm = event.comm.decode('utf-8', 'replace')

    print(f"Перехват! PID: {pid}, UID: {uid}, "
          f"Процесс: {comm}")

# Подписываем функцию print_event на события из кольцевого буфера 'events'
b["events"].open_perf_buffer(print_event)

# Бесконечный цикл опроса буфера
while True:
    try:
        b.perf_buffer_poll()
    except KeyboardInterrupt:
        print("\n[*] Завершение работы монитора.")
        exit()
```

Программа запускается с правами суперпользователя:

sudo python3 monitor_template.py

```
boris_zhuk@DESKTOP-D73BTJH:/mnt/d/programming/CM3KC/SMZKS-2026-autumn/labs/lab3$ sudo python3 monitor_template.py
In file included from <built-in>:2:
In file included from /virtual/include/bcc/bpf.h:12:
In file included from include/linux/types.h:6:
In file included from include/uapi/linux/types.h:14:
In file included from include/uapi/linux/posix_types.h:5:
In file included from include/linux/stddef.h:5:
In file included from include/uapi/linux/stddef.h:5:
In file included from include/linux/compiler_types.h:108:
include/linux/compiler-clang.h:49:9: warning: '__HAVE_BUILTIN_BSWAP32__' macro redefined [-Wmacro-redefined]
#define __HAVE_BUILTIN_BSWAP32__
```

Для проверки работы мониторинга во втором терминале были выполнены команды **cat**, **whoami** и **ls**.

```
boris_zhuk@DESKTOP-D73BTJH:/mnt/d/programming/CM3KC/SMZKS-2026-autumn/labs/lab3$ ls
monitor_template.py  theory.md
boris_zhuk@DESKTOP-D73BTJH:/mnt/d/programming/CM3KC/SMZKS-2026-autumn/labs/lab3$ cat /etc/hostname
DESKTOP-D73BTJH
boris_zhuk@DESKTOP-D73BTJH:/mnt/d/programming/CM3KC/SMZKS-2026-autumn/labs/lab3$ whoami
boris_zhuk
```

В результате запуска команд монитор зафиксировал соответствующие вызовы **execve**:

```
PID: 23425, UID: 1000, Процесс: /usr/bin/ls
PID: 23426, UID: 1000, Процесс: /usr/bin/cat
PID: 23427, UID: 1000, Процесс: /usr/bin/whoami
```

Задание 2. Динамический анализ аномального поведения (Индивидуальный вариант)

Целевой системный вызов и условие аномального поведения

Согласно индивидуальному варианту 6 в качестве целевого системного вызова выбран **sys_enter_mount**. Аномальным поведением считается попытка монтирования файловой системы без флага MS_RDONLY.

eBPF-зонд

Для перехвата системного вызова используется tracepoint sys_enter_mount. В ядре извлекаются следующие данные:

- PID процесса;
- UID пользователя;
- имя процесса;
- источник монтирования dev_name;
- целевой каталог dir_name;
- тип файловой системы type;
- флаги монтирования flags.

Для хранения данных используется BPF_PERCPU_ARRAY, что позволяет избежать превышения ограничения размера стека eBPF при работе с большой структурой данных.

Листинг программы:

```
#!/usr/bin/python3
from bcc import BPF
import time

bpf_text = """
#include <uapi/linux/ptrace.h>
#define MS_RDONLY 1

struct data_t {
   u32 pid;
   u32 uid;
   u64 flags;

   char comm[256];
   char source[256];
   char target[256];
   char fstype[64];
};

BPF_PERF_OUTPUT(events);
BPF_PERCPU_ARRAY(data_heap, struct data_t, 1);

// Перехват системного вызова mount
TRACEPOINT_PROBE(syscalls, sys_enter_mount) {
   u32 zero = 0;

   struct data_t *data = data_heap.lookup(&zero);
```

```
    if (!data) {
        return 0;
    }

    // Очистка структуры
    __builtin_memset(data, 0, sizeof(*data));

    data->flags = args->flags;

    // ФИЛЬТРАЦИЯ АНОМАЛИЙ В ЯДРЕ
    // Если установлен MS_RDONLY, событие игнорируется
    if (data->flags & MS_RDONLY) {
        return 0;
    }

    data->pid = bpf_get_current_pid_tgid() >> 32;
    data->uid = bpf_get_current_uid_gid();

    bpf_get_current_comm(&data->comm, sizeof(data->comm));

    // Получение аргументов системного вызова
    bpf_probe_read_user_str(
        &data->source,
        sizeof(data->source),
        args->dev_name
    );

    bpf_probe_read_user_str(
        &data->target,
        sizeof(data->target),
        args->dir_name
    );

    bpf_probe_read_user_str(
        &data->fstype,
        sizeof(data->fstype),
        args->type
    );

    // Отправка только аномальных событий
    events.perf_submit(args, data, sizeof(*data));

    return 0;
}
"""

b = BPF(text=bpf_text)

print("[*] Мониторинг запущен. Ожидание вызовов execve... (Нажмите Ctrl+C для выхода)")

def print_event(cpu, data, size):
    event = b["events"].event(data)
```

```
    pid = event.pid
    uid = event.uid

    comm = event.comm.decode('utf-8', 'replace').rstrip('\x00')
    source = event.source.decode('utf-8', 'replace').rstrip('\x00')
    target = event.target.decode('utf-8', 'replace').rstrip('\x00')
    fstype = event.fstype.decode('utf-8', 'replace').rstrip('\x00')

    flags = event.flags

    print("\n[!] ОБНАРУЖЕНА АНОМАЛИЯ!")
    print(f"PID: {pid}")
    print(f"UID: {uid}")
    print(f"Процесс: {comm}")
    print(f"Источник: {source}")
    print(f"Целевой каталог: {target}")
    print(f"Тип файловой системы: {fstype}")
    print(f"Флаги монтирования: {flags}")
    print("Причина: монтирование без MS_RDONLY")

b["events"].open_perf_buffer(print_event)

while True:
    try:
        b.perf_buffer_poll()
    except KeyboardInterrupt:
        print("\n[*] Завершение работы монитора.")
        exit()
```

Фильтрация событий на стороне ядра

Фильтрация выполняется непосредственно внутри eBPF-программы. Событие передаётся через **BPF_PERF_OUTPUT** только при выполнении условия: отсутствие флага **MS_RDONLY** в параметрах системного вызова mount.

```
        if (data->flags & MS_RDONLY) {
            return 0;
        }

        events.perf_submit(args, data, sizeof(*data));
```

Вызовы mount, содержащие флаг MS_RDONLY, не передаются в Python-скрипт. Это позволяет выполнять фильтрацию на стороне ядра и не передавать в пространство пользователя все события системного вызова, а только попытки монтирования без указанного флага.

Генерация и перехват аномального поведения

Для проверки работы зонда была создана директория /mnt/test и выполнена попытка монтирования файловой системы tmpfs без флага MS_RDONLY.

```
boris_zhuk@DESKTOP-D73BTJH:/mnt/d/programming/CM3KC/SMZKS-2026-autumn/labs/lab3$ sudo mkdir -p /mnt/test
boris_zhuk@DESKTOP-D73BTJH:/mnt/d/programming/CM3KC/SMZKS-2026-autumn/labs/lab3$ sudo mount -t tmpfs /mnt/test
```

В результате eBPF-зонд обнаружил попытку монтирования файловой системы без флага MS_RDONLY и сформировал предупреждение.

```
boris_zhuk@DESKTOP-D73BTJH:/mnt/d/programming/CM3KC/SMZKS-2026-autumn/labs/lab3$ sudo python3 monitor_template.py
In file included from <built-in>:2:
In file included from /virtual/include/bcc/bpf.h:12:
In file included from include/linux/types.h:6:
In file included from include/uapi/linux/types.h:14:
In file included from include/uapi/linux/posix_types.h:5:
In file included from include/linux/stddef.h:5:
In file included from include/uapi/linux/stddef.h:5:
In file included from include/linux/compiler_types.h:108:
include/linux/compiler-clang.h:49:9: warning: '__HAVE_BUILTIN_BSWAP32__' macro redefined [-Wmacro-redefined]
#define __HAVE_BUILTIN_BSWAP32__
```

```
[!] ОБНАРУЖЕНА АНОМАЛИЯ!
PID: 24121
UID: 0
Процесс: mount
Источник: tmpfs
Целевой каталог: /mnt/d/programming/CM3KC/SMZKS-2026-autumn/labs/lab3/mnt/test
Тип файловой системы: tmpfs
Флаги монтирования: 0
Причина: монтирование без MS_RDONLY
```

Дополнительно была выполнена проверка монтирования файловой системы с установленным флагом MS_RDONLY. При этом предупреждение об аномалии не появилось, поскольку в параметрах системного вызова был установлен флаг MS_RDONLY. Это подтверждает корректную работу фильтрации на стороне ядра.

Задание 3. Интеграция с пространством пользователя и аудит

Python-скрипт из задания 2 был расширен функцией автоматической записи обнаруженных аномалий в локальный файл аудита security_audit.log. При получении события от eBPF-программы Python-скрипт формирует запись, содержащую временную метку, PID и UID процесса, имя процесса, параметры монтирования и причину обнаружения аномалии.

Листинг программы:

```
#!/usr/bin/python3
from bcc import BPF
import time
from datetime import datetime

bpf_text = """
#include <uapi/linux/ptrace.h>
#define MS_RDONLY 1

struct data_t {
   u32 pid;
   u32 uid;
   u64 flags;

   char comm[256];
   char source[256];
```

```
    char target[256];
    char fstype[64];
};

BPF_PERF_OUTPUT(events);
BPF_PERCPU_ARRAY(data_heap, struct data_t, 1);

TRACEPOINT_PROBE(syscalls, sys_enter_mount) {
    u32 zero = 0;

    struct data_t *data = data_heap.lookup(&zero);

    if (!data) {
        return 0;
    }

    __builtin_memset(data, 0, sizeof(*data));

    data->flags = args->flags;

    if (data->flags & MS_RDONLY) {
        return 0;
    }

    data->pid = bpf_get_current_pid_tgid() >> 32;
    data->uid = bpf_get_current_uid_gid();

    bpf_get_current_comm(&data->comm, sizeof(data->comm));

    bpf_probe_read_user_str(
        &data->source,
        sizeof(data->source),
        args->dev_name
    );

    bpf_probe_read_user_str(
        &data->target,
        sizeof(data->target),
        args->dir_name
    );

    bpf_probe_read_user_str(
        &data->fstype,
        sizeof(data->fstype),
        args->type
    );

    events.perf_submit(args, data, sizeof(*data));

    return 0;
}
"""
```

```
b = BPF(text=bpf_text)

print("[*] Мониторинг запущен. Ожидание вызовов execve... (Нажмите Ctrl+C для выхода)")

def print_event(cpu, data, size):
    event = b["events"].event(data)

    pid = event.pid
    uid = event.uid

    comm = event.comm.decode('utf-8', 'replace').rstrip('\x00')
    source = event.source.decode('utf-8', 'replace').rstrip('\x00')
    target = event.target.decode('utf-8', 'replace').rstrip('\x00')
    fstype = event.fstype.decode('utf-8', 'replace').rstrip('\x00')

    flags = event.flags

    print("\n[!] ОБНАРУЖЕНА АНОМАЛИЯ!")
    print(f"PID: {pid}")
    print(f"UID: {uid}")
    print(f"Процесс: {comm}")
    print(f"Источник: {source}")
    print(f"Целевой каталог: {target}")
    print(f"Тип файловой системы: {fstype}")
    print(f"Флаги монтирования: {flags}")
    print("Причина: монтирование без MS_RDONLY")

    # Получение текущей временной метки
    timestamp = datetime.now().astimezone().isoformat(
        timespec="seconds"
    )

    # Формирование записи журнала
    log_entry = (
        f"[{timestamp}] "
        f"PID: {pid} | "
        f"UID: {uid} | "
        f"Процесс: {comm} | "
        f"Источник: {source} | "
        f"Целевой каталог: {target} | "
        f"Тип файловой системы: {fstype} | "
        f"Флаги: {flags} | "
        f"Причина: монтирование без MS_RDONLY\n"
    )

    # Запись события в файл аудита
    with open("security_audit.log", "a", encoding="utf-8") as log:
        log.write(log_entry)

    print("[+] Событие записано в security_audit.log")
```

```
b["events"].open_perf_buffer(print_event)

while True:
    try:
        b.perf_buffer_poll()
    except KeyboardInterrupt:
        print("\n[*] Завершение работы монитора.")
        exit()
```

Для проверки была создана аномальная ситуация — выполнена попытка монтирования файловой системы tmpfs без флага MS_RDONLY.

```
boris_zhuk@DESKTOP-D73BTJH:/mnt/d/programming/CM3KC/SMZKS-2026-autumn/labs/lab3$ sudo mount -t tmpfs tmpfs ./mnt/test
```

После обнаружения события программа вывела предупреждение:

```
[!] ОБНАРУЖЕНА АНОМАЛИЯ!
PID: 24314
UID: 0
Процесс: mount
Источник: tmpfs
Целевой каталог: /mnt/d/programming/CM3KC/SMZKS-2026-autumn/labs/lab3/mnt/test
Тип файловой системы: tmpfs
Флаги монтирования: 0
Причина: монтирование без MS_RDONLY
[+] Событие записано в security_audit.log
```

Одновременно информация была автоматически записана в файл **security_audit.log**.

Содержимое файла аудита:

```
boris_zhuk@DESKTOP-D73BTJH:/mnt/d/programming/CM3KC/SMZKS-2026-autumn/labs/lab3$ cat security_audit.log
[2026-10-31T20:13:42+03:00] PID: 24314 | UID: 0 | Процесс: mount | Источник: tmpfs | Целевой каталог: /mnt/d/programming/CM3KC/SMZKS-2026-autu
mn/labs/lab3/mnt/test | Тип файловой системы: tmpfs | Флаги: 0 | Причина: монтирование без MS_RDONLY
```

Реализована интеграция eBPF-программы с пространством пользователя: обнаруженное в ядре аномальное событие передаётся Python-скрипту, после чего автоматически сохраняется в локальный журнал безопасности security_audit.log.

Задание 4. Проектирование профилей защиты (Без написания кода)

Сравнение Seccomp-BPF и eBPF (kprobes)

Seccomp-BPF и обычный **eBPF**-зонд решают разные задачи.

Seccomp-BPF

Предназначен для ограничения поведения процесса. С его помощью можно задать фильтр системных вызовов, которые разрешены или запрещены процессу. Например, можно разрешить только необходимые системные вызовы, а попытку выполнения запрещённого вызова заблокировать или завершить процесс.

eBPF с **kprobes/tracepoints**

В основном используется для наблюдения за работой ядра. С помощью eBPF можно перехватывать события, получать информацию о процессах и системных вызовах, анализировать параметры операций и передавать результаты в пространство пользователя.

В лабораторной работе eBPF использовался для мониторинга sys_enter_mount. Программа обнаруживала попытки монтирования файловых систем без флага MS_RDONLY и передавала информацию в Python-скрипт. При этом сама операция монтирования не блокировалась.

Характеристика	**Seccomp-BPF**	**eBPF**
Основное назначение	Ограничение системных вызовов	Мониторинг и анализ
Контроль системных вызовов	Да	Да, в основном наблюдение
Блокировка операции	Да, через заданные действия seccomp	Обычно нет
Анализ параметров	Ограниченный	Более гибкий
Использование в лабораторной	Не использовался	Использовался для обнаружения аномалий

Стандартный eBPF-зонд преимущественно используется для мониторинга, обнаружения и аудита, а не для блокировки действий вредоносного ПО. eBPF-программа может обнаружить подозрительное действие и сообщить о нём, но для непосредственного ограничения системных вызовов процесса существуют специализированные механизмы, например Seccomp-BPF.

Атака TOCTOU применительно к eBPF-зонду

TOCTOU (Time-of-Check to Time-of-Use) – это атака типа «время проверки – время использования». Она возникает, когда программа сначала проверяет состояние объекта, а затем через некоторое время использует этот объект. Между проверкой и использованием злоумышленник может изменить объект. Применительно к разработанному eBPF-зонду проблема может возникнуть следующим образом. Зонд получает информацию о вызове mount и считывает его параметры, включая источник монтирования, целевой каталог и тип файловой системы. Между моментом считывания этих данных и фактическим использованием параметров ядром пользовательский процесс потенциально может изменить содержимое переданных указателей. В результате информация, проанализированная зондом, может отличаться от фактически использованных параметров системного вызова. Особенно важно, что наш зонд предназначен для наблюдения, а не для принятия решения о блокировке операции. Поэтому между моментом обнаружения события и реакцией Python-скрипта может пройти некоторое время. За этот промежуток состояние файла уже может измениться. Для противодействия TOCTOU необходимо не разделять проверку и критическую операцию, а выполнять проверку и принятие решения

атомарно в ядре. Для операций с файлами можно использовать атомарные системные механизмы и ограничения доступа, а для контроля системных вызовов – применять Seccomp-BPF или другой механизм принудительного контроля. Для рассматриваемой архитектуры рациональным вариантом является сочетание механизмов:

eBPF – обнаружение и аудит – система реагирования

и

Seccomp/механизмы контроля доступа – предотвращение запрещённой операции.

eBPF отвечает за наблюдение и регистрацию событий, а механизм принудительного контроля используется непосредственно для предотвращения опасных действий.

Вывод: практически освоил методы разработки программ расширенного фильтра пакетов Беркли (eBPF) на языке C для мониторинга событий уровня ядра, динамического перехвата системных вызовов (kprobes/tracepoints) и выявления аномального поведения процессов в операционной системе Linux с использованием фреймворка BCC (BPF Compiler Collection).